

LAPORAN SINGKAT PRAKTIKUM

1.1 Source Code Java

```
1 package pekan9_2511533020;
2 import javax.swing.*;
3
4
5
6
7 public class PetaKampusFT_2511533020 extends JFrame {
8
9
10     private Map<String, List<String>> adjMatrix_3020;
11     private Map<String, Point> nodePositions_3020;
12     private List<String> visitedNodes_3020;
13     private List<String> pathFound_3020;
14     private String currentMethod_3020 = "";
15
16
17     private JComboBox<String> cmbStart_3020;
18     private JComboBox<String> cmbGoal_3020;
19     private JButton btnBFS_3020, btnDFS_3020, btnReset_3020;
20     private JLabel lblHasil_3020, lblJalur_3020, lblDikunjungi_3020, lblJumlah_3020;
21     private GraphPanel_3020 panelGraph_3020;
22
23     public PetaKampusFT_2511533020() {
24         setTitle("Pencarian Jalur FT UNAND - 2511533020");
25         setSize(850, 600);
26         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
27         setLocationRelativeTo(null);
28
29
30         JPanel mainPanel_3020 = new JPanel();
31         mainPanel_3020.setLayout(null);
32         mainPanel_3020.setBackground(new Color(235, 240, 245));
33         setContentPane(mainPanel_3020);
34
35
36         initGraphData_3020();
37
38
39         JPanel banner_3020 = new JPanel(null);
40         banner_3020.setBackground(new Color(27, 79, 114));
41         banner_3020.setBounds(0, 0, 850, 45);
42         JLabel lblTitle_3020 = new JLabel("PENCARIAN JALUR MENGGUNAKAN BFS DAN DFS", JLabel.CENTER);
43         lblTitle_3020.setFont(new Font("SansSerif", Font.BOLD, 16));
44         lblTitle_3020.setForeground(Color.WHITE);
45         lblTitle_3020.setBounds(0, 0, 850, 45);
46         banner_3020.add(lblTitle_3020);
47         mainPanel_3020.add(banner_3020);
```

```

50     JLabel lblStart = new JLabel("Lokasi Awal :");
51     lblStart.setFont(new Font("SansSerif", Font.PLAIN, 13));
52     lblStart.setBounds(20, 60, 90, 25);
53     mainPanel_3020.add(lblStart);
54
55     String[] nodesArray = nodePositions_3020.keySet().toArray(new String[0]);
56     cmbStart_3020 = new JComboBox<>(nodesArray);
57     cmbStart_3020.setBounds(115, 60, 130, 25);
58     mainPanel_3020.add(cmbStart_3020);
59
60     JLabel lblGoal = new JLabel("Lokasi Tujuan :");
61     lblGoal.setFont(new Font("SansSerif", Font.PLAIN, 13));
62     lblGoal.setBounds(20, 95, 90, 25);
63     mainPanel_3020.add(lblGoal);
64
65     cmbGoal_3020 = new JComboBox<>(nodesArray);
66     cmbGoal_3020.setBounds(115, 95, 130, 25);
67     mainPanel_3020.add(cmbGoal_3020);
68
69
70     btnBFS_3020 = new JButton("[ BFS ]");
71     btnBFS_3020.setBackground(new Color(46, 204, 113));
72     btnBFS_3020.setForeground(Color.BLACK);
73     btnBFS_3020.setFont(new Font("SansSerif", Font.BOLD, 12));
74     btnBFS_3020.setBounds(480, 75, 90, 30);
75     mainPanel_3020.add(btnBFS_3020);
76
77
78     btnDFS_3020 = new JButton("[ DFS ]");
79     btnDFS_3020.setBackground(new Color(244, 156, 23));
80     btnDFS_3020.setForeground(Color.BLACK);
81     btnDFS_3020.setFont(new Font("SansSerif", Font.BOLD, 12));
82     btnDFS_3020.setBounds(585, 75, 90, 30);
83     mainPanel_3020.add(btnDFS_3020);
84
85
86     btnReset_3020 = new JButton("[ RESET ]");
87     btnReset_3020.setBackground(new Color(231, 76, 60));
88     btnReset_3020.setForeground(Color.WHITE);
89     btnReset_3020.setFont(new Font("SansSerif", Font.BOLD, 12));
90     btnReset_3020.setBounds(690, 75, 100, 30);
91     mainPanel_3020.add(btnReset_3020);
92

```

```

94     panelGraph_3020 = new GraphPanel_3020();
95     panelGraph_3020.setBounds(20, 140, 795, 270);
96     mainPanel_3020.add(panelGraph_3020);
97
98
99     lblHasil_3020 = new JLabel("Hasil Pencarian :");
100    lblHasil_3020.setFont(new Font("SansSerif", Font.BOLD, 13));
101    lblHasil_3020.setBounds(20, 425, 600, 20);
102    mainPanel_3020.add(lblHasil_3020);
103
104    lblJalur_3020 = new JLabel("Jalur :");
105    lblJalur_3020.setFont(new Font("SansSerif", Font.BOLD, 13));
106    lblJalur_3020.setBounds(20, 445, 790, 20);
107    mainPanel_3020.add(lblJalur_3020);
108
109    lblDikunjungi_3020 = new JLabel("Node Dikunjungi :");
110    lblDikunjungi_3020.setFont(new Font("SansSerif", Font.BOLD, 13));
111    lblDikunjungi_3020.setBounds(20, 465, 790, 20);
112    mainPanel_3020.add(lblDikunjungi_3020);
113
114    lblJumlah_3020 = new JLabel("Jumlah Node Dikunjungi : 0");
115    lblJumlah_3020.setFont(new Font("SansSerif", Font.BOLD, 13));
116    lblJumlah_3020.setBounds(20, 485, 400, 20);
117    mainPanel_3020.add(lblJumlah_3020);
118
119
120    btnBFS_3020.addActionListener(e -> BFS());
121    btnDFS_3020.addActionListener(e -> DFS());
122    btnReset_3020.addActionListener(e -> resetGraph());
123
124
125    private void initGraphData_3020() {
126        adjMatrix_3020 = new LinkedHashMap<>();
127        nodePositions_3020 = new LinkedHashMap<>();
128        visitedNodes_3020 = new ArrayList<>();
129        pathFound_3020 = new ArrayList<>();
130
131
132        nodePositions_3020.put("Dekanat", new Point(60, 60));
133        nodePositions_3020.put("Teknik Sipil", new Point(340, 60));
134        nodePositions_3020.put("Teknik Mesin", new Point(620, 60));
135
136        nodePositions_3020.put("Gedung A", new Point(60, 130));
137        nodePositions_3020.put("Teknik Lingkungan", new Point(340, 130));

```

```

138     nodePositions_3020.put("Teknik_Elektro", new Point(620, 130));
139
140     nodePositions_3020.put("Gedung_B", new Point(60, 200));
141     nodePositions_3020.put("Lab_Terpadu", new Point(340, 200));
142     nodePositions_3020.put("Masjid_Teknik", new Point(620, 200));
143
144     nodePositions_3020.put("Teknik_Industri", new Point(340, 250));
145
146     for (String node : nodePositions_3020.keySet()) {
147         adjMatrix_3020.put(node, new ArrayList<>());
148     }
149
150     addEdge_3020("Dekanat", "Gedung_A");
151     addEdge_3020("Dekanat", "Teknik_Sipil");
152     addEdge_3020("Dekanat", "Teknik_Mesin");
153     addEdge_3020("Gedung_A", "Gedung_B");
154     addEdge_3020("Gedung_A", "Teknik_Industri");
155     addEdge_3020("Gedung_B", "Lab_Terpadu");
156     addEdge_3020("Gedung_B", "Teknik_Mesin");
157     addEdge_3020("Teknik_Sipil", "Teknik_Mesin");
158     addEdge_3020("Teknik_Sipil", "Lab_Terpadu");
159     addEdge_3020("Teknik_Mesin", "Teknik_Elektro");
160     addEdge_3020("Teknik_Elektro", "Teknik_Industri");
161     addEdge_3020("Teknik_Elektro", "Masjid_Teknik");
162     addEdge_3020("Teknik_Industri", "Teknik_Lingkungan");
163     addEdge_3020("Teknik_Lingkungan", "Teknik_Sipil");
164     addEdge_3020("Lab_Terpadu", "Masjid_Teknik");
165     addEdge_3020("Masjid_Teknik", "Teknik_Lingkungan");
166 }
167
168 private void addEdge_3020(String u, String v) {
169     adjMatrix_3020.get(u).add(v);
170     adjMatrix_3020.get(v).add(u);
171 }
172
173 public void BFS() {
174     visitedNodes_3020.clear();
175     pathFound_3020.clear();
176     currentMethod_3020 = "BFS";
177
178     String start = (String) cmbStart_3020.getSelectedItemAt();
179     String goal = (String) cmbGoal_3020.getSelectedItemAt();

```

```

183     Queue<String> queue = new LinkedList<>();
184     Map<String, String> parentMap = new HashMap<>();
185
186     queue.add(start);
187     visitedNodes_3020.add(start);
188
189     boolean found = false;
190     while (!queue.isEmpty()) {
191         String current = queue.poll();
192         if (current.equals(goal)) {
193             found = true;
194             break;
195         }
196         for (String neighbor : adjMatrix_3020.get(current)) {
197             if (!visitedNodes_3020.contains(neighbor)) {
198                 visitedNodes_3020.add(neighbor);
199                 parentMap.put(neighbor, current);
200                 queue.add(neighbor);
201             }
202         }
203     }
204
205     if (found) {
206         String curr = goal;
207         while (curr != null) {
208             pathFound_3020.add(0, curr);
209             curr = parentMap.get(curr);
210         }
211         displayPath();
212         displayGraph();
213     }
214 }
215
216 public void DFS() {
217     visitedNodes_3020.clear();
218     pathFound_3020.clear();
219     currentMethod_3020 = "DFS";
220
221     String start = (String) cmbStart_3020.getSelectedItemAt();
222     String goal = (String) cmbGoal_3020.getSelectedItemAt();
223
224     Stack<String> stack = new Stack<>();
225     Map<String, String> parentMap = new HashMap<>();

```

```

227
228     stack.push(start);
229
230     boolean found = false;
231     while (!stack.isEmpty()) {
232         String current = stack.pop();
233
234         if (!visitedNodes_3020.contains(current)) {
235             visitedNodes_3020.add(current);
236         }
237
238         if (current.equals(goal)) {
239             found = true;
240             break;
241         }
242
243         List<String> neighbors = adjMatrix_3020.get(current);
244         for (int i = neighbors.size() - 1; i >= 0; i--) {
245             String neighbor = neighbors.get(i);
246             if (!visitedNodes_3020.contains(neighbor) && !stack.contains(neighbor)) {
247                 parentMap.put(neighbor, current);
248                 stack.push(neighbor);
249             }
250         }
251     }
252
253     if (found) {
254         String curr = goal;
255         while (curr != null) {
256             pathFound_3020.add(0, curr);
257             curr = parentMap.get(curr);
258         }
259     }
260     displayPath();
261     displayGraph();
262 }
263
264
265 public void displayGraph() {
266     panelGraph_3020.repaint();
267 }
268
269
270 public void displayPath() {

```

```

271     lblHasil_3020.setText("Hasil Pencarian : " + currentMethod_3020);
272     if (pathFound_3020.isEmpty()) {
273         lblJalur_3020.setText("Jalur : Tidak ditemukan");
274     } else {
275         lblJalur_3020.setText("Jalur : " + String.join(" -> ", pathFound_3020));
276     }
277     lblDikunjungi_3020.setText("Node Dikunjungi : " + String.join(", ", visitedNodes_3020));
278     lblJumlah_3020.setText("Jumlah Node Dikunjungi : " + visitedNodes_3020.size());
279 }
280
281
282 public void resetGraph() {
283     visitedNodes_3020.clear();
284     pathFound_3020.clear();
285     currentMethod_3020 = "";
286     lblHasil_3020.setText("Hasil Pencarian :");
287     lblJalur_3020.setText("Jalur :");
288     lblDikunjungi_3020.setText("Node Dikunjungi :");
289     lblJumlah_3020.setText("Jumlah Node Dikunjungi : 0");
290     displayGraph();
291 }
292
293
294 private class GraphPanel_3020 extends JPanel {
295     public GraphPanel_3020() {
296         setBackground(Color.WHITE);
297         setBorder(BorderFactory.createLineBorder(Color.GRAY, 1));
298     }
299
300     @Override
301     protected void paintComponent(Graphics g) {
302         super.paintComponent(g);
303         Graphics2D g2 = (Graphics2D) g;
304         g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
305         g2.setColor(Color.BLACK);
306         g2.setFont(new Font("Monospaced", Font.BOLD, 13));
307         g2.drawString("VISUALISASI GRAF", 15, 25);
308
309         float[] dashPattern = {6.0f, 5.0f};
310         Stroke dashedStroke = new BasicStroke(1.5f, BasicStroke.CAP_BUTT, BasicStroke.JOIN_MITER, 10.0f, dashPattern, 0.0f);
311         Stroke solidStroke = new BasicStroke(2.5f);
312
313         Set<String> drawEdges = new HashSet<>();
314         for (String node : adjMatrix_3020.keySet()) {

```

```

315         Point p1 = nodePositions_3020.get(node);
316         for (String neighbor : adjMatrix_3020.get(node)) {
317             String edgeKey = node.compareTo(neighbor) < 0 ? node + "-" + neighbor : neighbor + "-" + node;
318             if (!drawnEdges.contains(edgeKey)) {
319                 Point p2 = nodePositions_3020.get(neighbor);
320
321                 if (pathFound_3020.contains(node) && pathFound_3020.contains(neighbor) &&
322                     Math.abs(pathFound_3020.indexOf(node) - pathFound_3020.indexOf(neighbor)) == 1) {
323                     g2.setColor(new Color(231, 76, 60));
324                     g2.setStroke(solidStroke);
325                 } else {
326                     g2.setColor(Color.LIGHT_GRAY);
327                     g2.setStroke(dashedStroke);
328                 }
329                 g2.drawLine(p1.x, p1.y, p2.x, p2.y);
330                 drawnEdges.add(edgeKey);
331             }
332         }
333     }
334
335     g2.setFont(new Font("Monospaced", Font.BOLD, 12));
336     for (Map.Entry<String, Point> entry : nodePositions_3020.entrySet()) {
337         String nodeName = entry.getKey();
338         Point p = entry.getValue();
339
340         if (pathFound_3020.contains(nodeName)) {
341             g2.setColor(new Color(41, 128, 185));
342         } else if (visitedNodes_3020.contains(nodeName)) {
343             g2.setColor(new Color(211, 84, 0));
344         } else {
345             g2.setColor(Color.BLACK);
346         }
347
348         FontMetrics fm = g2.getFontMetrics();
349         int textwidth = fm.stringWidth(nodeName);
350         int textHeight = fm.getHeight();
351         g2.clearRect(p.x - (textwidth / 2) - 4, p.y - (textHeight / 2), textwidth + 8, textHeight);
352
353         g2.drawString(nodeName, p.x - (textwidth / 2), p.y + (textHeight / 4));
354     }
355 }
356
357 public static void main(String[] args) {

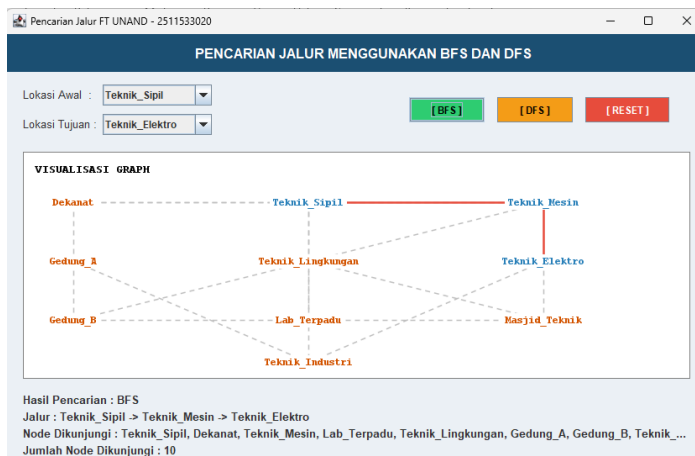
```

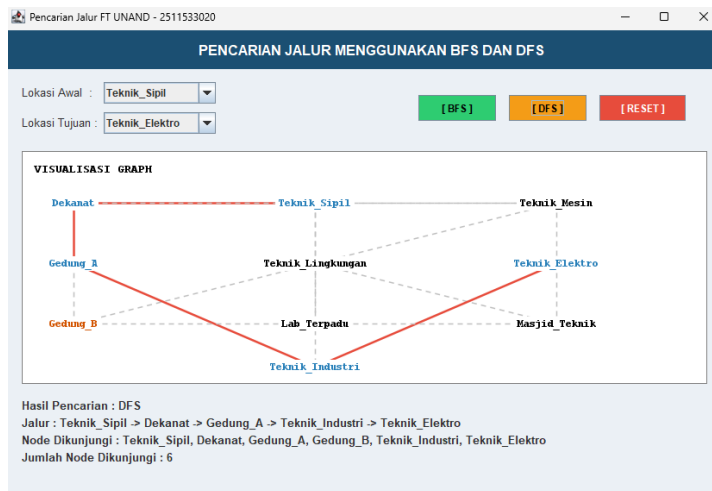
```

359     SwingUtilities.invokeLater(() -> {
360         new PetaKampusFT_2511533020().setVisible(true);
361     });
362 }
363 }

```

1.2 Screenshot Program Saat Di Run





1.3 Deskripsi Peta

Dalam tugas ini, peta lokasi yang saya buat mengambil studi kasus nyata dari lingkungan Fakultas Teknik Universitas Andalas (FT Unand). Peta ini memodelkan letak gedung-gedung jurusan dan fasilitas utama yang sering dikunjungi mahasiswa. Model yang digunakan adalah *undirected graph* (graf tidak berarah), artinya jalan atau koridor yang menghubungkan satu gedung ke gedung lain bisa dilewati dua arah secara bebas (bolak-balik) tanpa ada batasan satu jalur.

1.4 Jumlah Node dan Edge

Graf yang saya buat untuk memenuhi tugas ini berlokasi di area Fakultas Teknik Unand dan juga ini sudah memenuhi syarat minimal tugas karena memiliki komponen sebagai berikut:

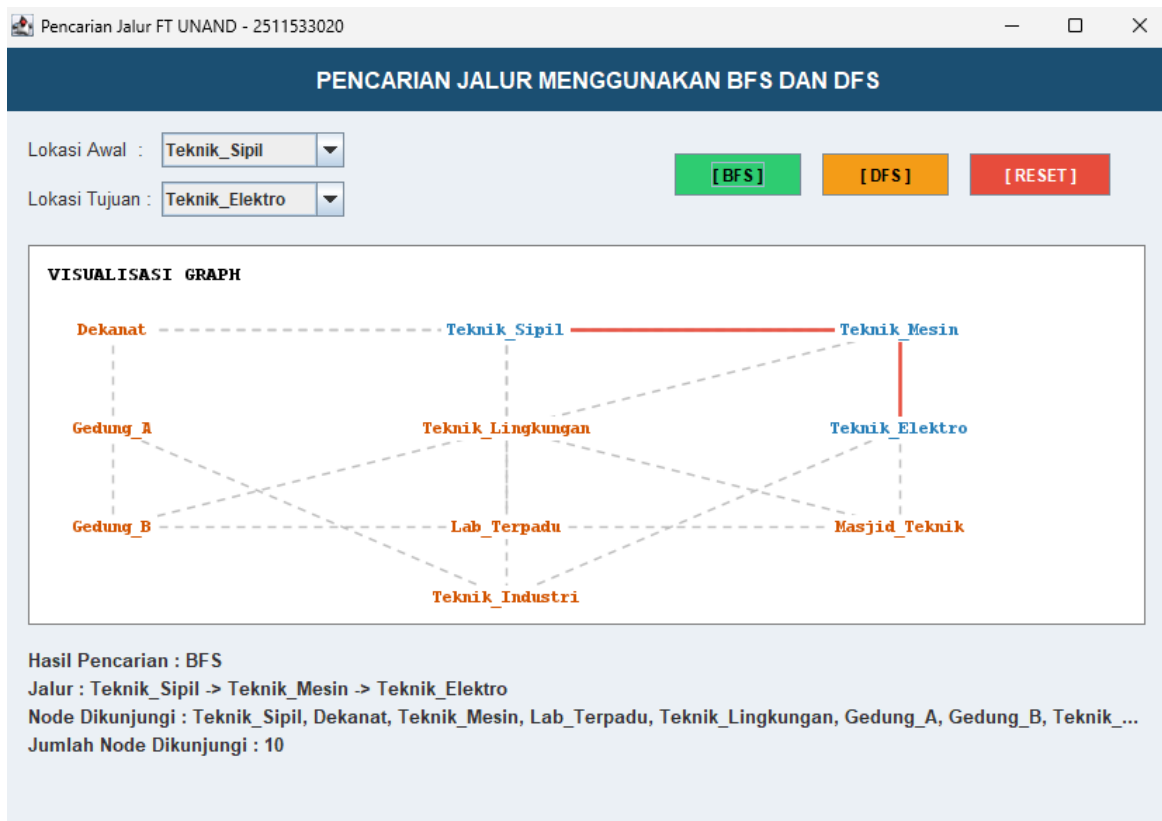
- **Jumlah Node (Simpul):** 10 Node.

Gedung yang dimasukkan meliputi Dekanat, Teknik Sipil, Teknik Mesin, Teknik Elektro, Gedung A, Teknik Lingkungan, Gedung B, Lab Terpadu, Masjid Teknik, dan Teknik Industri.

- **Jumlah Edge (Sisi):** 16 Edge.

Jalur penghubungnya meliputi Dekanat ke Gedung A, Dekanat ke Teknik Sipil, Dekanat ke Teknik Mesin, Gedung A ke Gedung B, Gedung A ke Teknik Industri, Gedung B ke Lab Terpadu, Gedung B ke Teknik Mesin, Teknik Sipil ke Teknik Mesin, Teknik Sipil ke Lab Terpadu, Teknik Mesin ke Teknik Elektro, Teknik Elektro ke Teknik Industri, Teknik Elektro ke Masjid Teknik, Teknik Industri ke Teknik Lingkungan, Teknik Lingkungan ke Teknik Sipil, Lab Terpadu ke Masjid Teknik, dan Masjid Teknik ke Teknik Lingkungan.

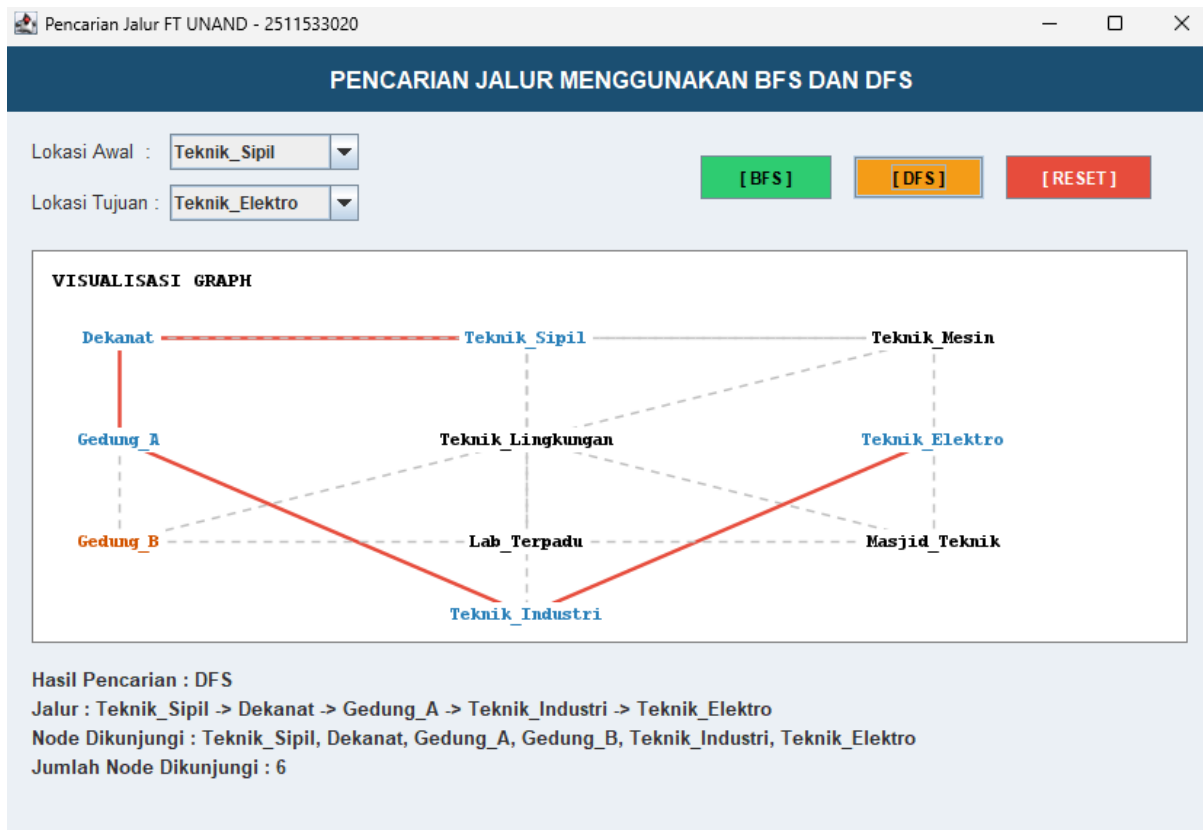
1.5 Hasil BFS



Ketika pengujian dilakukan menggunakan metode *Breadth-First Search* dengan menetapkan titik awal dari Teknik Sipil menuju titik tujuan Teknik Elektro, sistem berhasil menemukan jalur optimal yang terbentuk melalui rute Teknik_Sipil -> Teknik_Mesin -> Teknik_Elektro. Dalam proses pencarian ini, algoritma menyisir dan mengunjungi seluruh simpul yang ada di dalam graf dengan urutan yaitu Teknik Sipil, Dekanat, Teknik Mesin, Lab Terpadu, Teknik Lingkungan, Gedung A, Gedung B, Teknik Industri, Masjid Teknik, dan terakhir mengunci Teknik Elektro, sehingga total node yang dieksplorasi berjumlah sepuluh node.

Algoritma BFS beroperasi secara melebar menggunakan bantuan struktur data antrian (*queue*), di mana pencarian meronda seluruh node tetangga pada level yang sama secara horizontal terlebih dahulu sebelum melangkah lebih dalam, sehingga meskipun harus memeriksa seluruh simpul yang ada di peta, algoritma ini memberikan jaminan akurat bahwa jalur akhir yang terbentuk merupakan rute terpendek dengan jumlah lompatan terkecil (hanya melewati 2 sisi penghubung).

1.6 Hasil DFS



Sementara itu, ketika pengujian dilakukan menggunakan metode *Depth-First Search* dengan titik awal dan tujuan yang sama, sistem menghasilkan jalur akhir yang melewati rute Teknik_Sipil -> Dekanat -> Gedung_A -> Teknik_Industri -> Teknik_Elektro. Melalui metode ini, urutan simpul yang sempat dikunjungi oleh program tercatat melalui rute Teknik Sipil, Dekanat, Gedung A, Gedung B, Teknik Industri, dan langsung berhenti saat menemukan Teknik Elektro, dengan jumlah keseluruhan node yang dieksplorasi adalah sebanyak enam node.

Algoritma DFS bekerja dengan prinsip menelusuri sedalam mungkin ke dalam satu cabang jalur menggunakan bantuan struktur data tumpukan (*stack*), di mana program akan terus menelusuri rute ke depan melewati area Dekanat dan gedung perkuliahan umum terlebih dahulu. Begitu target ditemukan di ujung cabang penelusuran tersebut, DFS langsung menghentikan pencarian tanpa perlu memeriksa sisa node lainnya, yang membuat jumlah eksplorasinya lebih sedikit daripada BFS meskipun jalur jalannya jadi memutar lebih jauh.

1.7 Kesimpulan Perbandingan BFS dan DFS

Dari hasil pengujian dan analisis yang dilakukan pada peta Fakultas Teknik Universitas Andalas ini, kelihatan jelas kalau kedua algoritma punya cara kerja yang bertolak belakang

dalam mencari jalur. Algoritma BFS terasa sangat diandalkan kalau kita ingin mencari rute yang paling dekat atau optimal. Karena sifatnya yang menyisir semua tetangga terdekat dulu secara merata, BFS sukses menemukan jalur terpendek (Teknik_Sipil -> Teknik_Mesin -> Teknik_Elektro), walaupun konsekuensinya program harus memeriksa seluruh sepuluh node yang ada di peta sehingga proses eksplorasinya jadi lebih banyak

.Sementara itu, algoritma DFS memang bisa menemukan lokasi tujuan dengan jumlah eksplorasi node yang lebih sedikit, yaitu cuma enam node saja karena dia langsung menusuk ke rute paling dalam. Tapi kelemahannya, jalur akhir yang dihasilkan DFS malah memutar jauh (harus lewat Dekanat, Gedung A, dan Teknik Industri dulu baru sampai ke Teknik Elektro) sehingga jalurnya jadi kurang efisien secara jarak nyata. Kesimpulannya, untuk pemodelan navigasi atau pencarian jalan di dunia nyata seperti peta kampus ini, algoritma BFS jauh lebih cocok dan ideal digunakan ketimbang DFS karena ada jaminan kalau rute yang diberikan adalah jalur yang paling cepat dan dekat.